

C++ 面向对象编程自学手册

前言

本手册基于 C++ 面向对象编程（OOP）的核心概念编写，旨在帮助初学者系统掌握类与对象的核心知识。内容涵盖从基础概念到实战应用，通过实例解析和步骤指导，逐步构建 OOP 思维与编程能力。

第 1 章 面向对象编程基础

1.1 过程性编程与 OOP 的区别

- 过程性编程**：以步骤为中心，将问题拆解为函数和数据，例如处理球队数据时，通过输入、计算、显示等函数分步实现。
- 面向对象编程（OOP）**：以数据为中心，将数据与操作数据的方法封装为 "对象"，例如用 "选手" 对象整合姓名、击球数等数据及计算命中率的方法。

对比维度	过程性编程	OOP
核心思想	步骤优先，数据独立	数据与方法绑定
代码组织	函数与数据分离	封装为类和对象
扩展性	修改需调整多个函数	通过对象接口修改，低耦合

1.2 OOP 的核心特性

- 抽象**：提取核心特征（如股票的公司名称、价格），忽略无关细节。
- 封装**：将数据（私有）与操作方法（公有）捆绑，仅通过接口访问数据。
- 继承**：从现有类派生新类，复用代码（后续章节详解）。
- 多态**：同一接口表现不同行为（后续章节详解）。
- 代码复用**：通过类和继承减少重复代码。

第 2 章 类与对象

2.1 类的定义

类是用户自定义的类型，包含**数据成员**（属性）和**成员函数**（方法）。

示例：Stock 类声明

```
class Stock {  
private: // 私有成员（仅类内访问）  
    std::string company; // 公司名称  
    long shares;        // 持股数量  
    double share_val;    // 每股价格  
    double total_val;    // 总价值  
public: // 公有成员（外部可访问）  
    void buy(long num, double price); // 买入股票  
    void sell(long num, double price); // 卖出股票  
    void update(double price);        // 更新价格  
    void show();                      // 显示信息  
};
```

2.2 类的访问控制

- **private（私有）**：仅类内成员可访问（数据隐藏的核心）。
- **public（公有）**：外部可通过对象调用（接口）。
- **protected（保护）**：用于继承（后续章节）。

示例：

外部代码无法直接修改 `shares`，需通过 `buy()` 或 `sell()` 方法：

```
Stock s;  
s.shares = 100; // 错误：私有成员不可直接访问  
s.buy(100, 50); // 正确：通过公有方法修改
```

2.3 对象的创建与使用

对象是类的实例，如同变量是类型的实例。

创建对象：

```
Stock apple; // 声明对象
Stock* ptr = new Stock(); // 动态创建对象（需用delete释放）
```

调用成员函数：

```
apple.buy(10, 150.5); // 调用buy方法
apple.show();        // 显示股票信息
```

第 3 章 类的实现

3.1 类声明与方法定义分离

- **类声明**：放在头文件（.h），描述接口。
- **方法定义**：放在源文件（.cpp），用::指定作用域。

示例（Stock 类）：

头文件stock.h：

```
#ifndef STOCK_H
#define STOCK_H
#include <string>
class Stock {
private:
    std::string company;
    long shares;
    double share_val;
    double total_val;
    void set_tot() { total_val = shares * share_val; } // 内联函数
public:
    void buy(long num, double price);
    void sell(long num, double price);
```

```
};  
#endif
```

源文件stock.cpp:

```
#include "stock.h"  
void Stock::buy(long num, double price) {  
    if (num > 0) {  
        shares += num;  
        share_val = price;  
        set_tot(); // 调用私有方法  
    }  
}
```

3.2 内联函数

- 定义在类声明中的函数自动为内联函数（如set_tot()）。
- 类外定义时需加inline关键字：

```
inline void Stock::update(double price) {  
    share_val = price;  
    set_tot();  
}
```

第 4 章 构造函数与析构函数

4.1 构造函数

用于初始化对象，与类名同名，无返回值。

示例：

```
class Stock {  
public:  
    // 带默认参数的构造函数  
    Stock(const std::string& co = "None", long n = 0, double pr = 0.0) {  
        company = co;
```

```
    shares = n;  
    share_val = pr;  
    set_tot();  
}  
};
```

对象初始化方式：

```
Stock s1("Apple", 100, 150.0); // 直接初始化  
Stock s2 = Stock("Banana", 50, 3.0); // 拷贝初始化（C++11支持列表初始化）
```

4.2 默认构造函数

- 无参数的构造函数，若未定义任何构造函数，编译器自动生成。
- 若定义了其他构造函数，需显式定义默认构造函数：

```
Stock::Stock() { // 默认构造函数  
    company = "None";  
    shares = 0;  
    share_val = 0.0;  
    total_val = 0.0;  
}
```

4.3 析构函数

对象销毁时自动调用，用于释放资源（如动态内存），名称为~类名。

示例：

```
class Stock {  
public:  
    ~Stock() {  
        // 若有动态分配的内存，需在此释放  
        std::cout << company << "对象已销毁\n";  
    }  
};
```

第 5 章 this 指针

- 指向当前对象的指针，隐含于所有非静态成员函数中。
- 用途：区分成员变量与参数，返回当前对象。

示例：

```
const Stock& Stock::topval(const Stock& s) const {  
    if (s.total_val > total_val)  
        return s;  
    else  
        return *this; // 返回当前对象  
}
```

第 6 章 对象数组

- 数组元素为对象，需调用构造函数初始化。

示例：

```
const int STKS = 2;  
Stock stocks[STKS] = {  
    Stock("Apple", 100, 150.0),  
    Stock("Banana", 50, 3.0)  
};  
// 访问数组元素  
stocks[0].show();
```

第 7 章 类作用域

- 类内名称（成员名）的作用域为整个类，外部需通过对象访问。
- **作用域内枚举（C++11）**：避免枚举量冲突：

```
enum class Egg { Small, Medium, Large };
enum class TShirt { Small, Medium, Large };
Egg e = Egg::Medium; // 需指定作用域
```

第 8 章 抽象数据类型 (ADT)

- 以通用方式描述数据及操作，隐藏实现细节（如栈、队列）。
- 示例：栈 (Stack) 类

```
// stack.h
class Stack {
private:
    enum { MAX = 10 };
    int items[MAX];
    int top;
public:
    Stack() { top = 0; }
    bool push(const int& item); // 入栈
    bool pop(int& item);       // 出栈
    bool isempty() const { return top == 0; }
};
```

实战练习

1. 设计一个 `BankAccount` 类，包含姓名、账号、余额，实现存款、取款、显示功能。
2. 用对象数组存储 3 个银行账户，计算总余额。
3. 实现一个栈类，用于存储整数，支持入栈、出栈和判空操作。

总结

本章从 OOP 基础到类的高级特性，逐步构建了面向对象编程的知识体系。核心在于理解**封装**（数据与方法绑定）、**继承**（代码复用）和**多态**（接口灵活），后续章节将深入探讨继承与多态。通过实例练习，可加深对类与对象的理解，为复杂程序设计奠定基础。

（注：文档部分内容可能由 AI 生成）